

Bilkent University

Department of Electrical and Electronics Engineering

EE-102: Introduction to Digital Circuit Design

Lab 2 Report: Introduction to VHDL

Author: Can Kurc

ID: 22502100

Date: February 12, 2026

1 Purpose of the Experiment

The purpose of this lab was to get familiar with the design, debugging and testing of combinational logic circuits using VHDL within the Vivado Integrated Design Environment (IDE). The lab objectives include:

- **VHDL Design and Implementation:** Learning how to modify and implement VHDL code, specifically focusing on submodules and top-level module integration.
- **Debugging Skills:** Identifying and fixing common syntax and logic errors (bugs) within a provided VHDL project.
- **Hardware Verification:** Demonstrating the functional circuit on a physical Basys 3 FPGA board by mapping code to hardware pins using a constraint file.
- **Simulation Techniques:** Writing a testbench to simulate inputs sequentially and verifying the output through waveform analysis.
- **Schematic Analysis:** Utilizing Vivado's tools to generate and compare RTL, synthesized, and implemented schematics to understand how the code translates to hardware.

2 Methodology

The experiment followed a structured digital design flow within the Vivado IDE to transition from VHDL code to a hardware implementation. The process followed four primary phases:

1. **Logic Customization:** This task required modifying the provided sub-module1.vhd file to implement specific combinational logic. Based on the last digit of my id number (0), the logic operators were kept as they are (AND, OR, and XOR gates).
2. **Debugging:** A “buggy” project was provided, containing six errors. These were identified and corrected by analyzing messages, checking port mappings and verifying physical pin assignments in the constraint file.
3. **Simulation:** To verify the logic before hardware development, a VHDL testbench (testbench.vhd) was created. This testbench simulated a binary counter to cycle all 256 possible input combinations (2^8) for the 8 switches, allowing for the observation of output waveforms in the Vivado simulator.
4. **Hardware Implementation:** Once the simulation was verified, the design was synthesized and implemented to map the logical gates to the FPGA's physical resources. A bitstream was then generated and loaded onto the Basys 3 board. The final verification was performed by toggling the switches and comparing the LED responses to the expected truth tables.

3 Task 1 and 2, Debugging and Modifying VHDL Code

The project began with a “buggy” VHDL design that required correction. The following steps were taken to make necessary changes and debug the VHDL code:

Upon first inspection of the project settings, it was discovered that the target board was not set to the Basys 3. After inspection of the texts on the Basys3 chip, the settings were changed to correspond to the right part number in Vivado.

Since my student id ended with a 0, there was no need to make any modifications to the sub-module1.vhd file (they were already set to AND, OR, and XOR).

3.1 Synthesis Debugging

After running synthesis, some critical errors were identified.

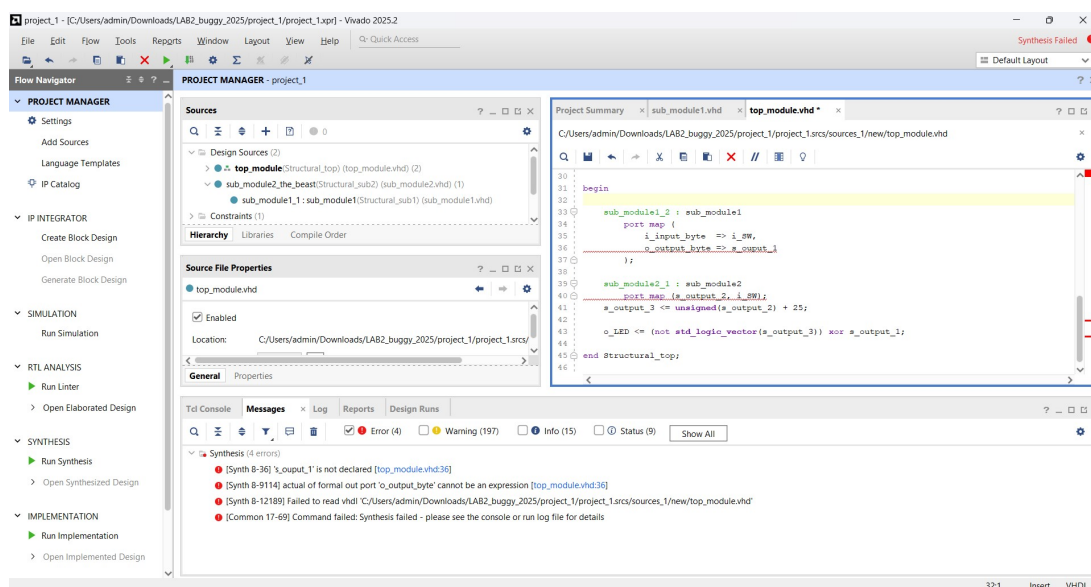


Figure 1: All initial errors

Typo Error: A typo was identified in the output signal name within top_module.vhd. This was corrected to match the port definition.

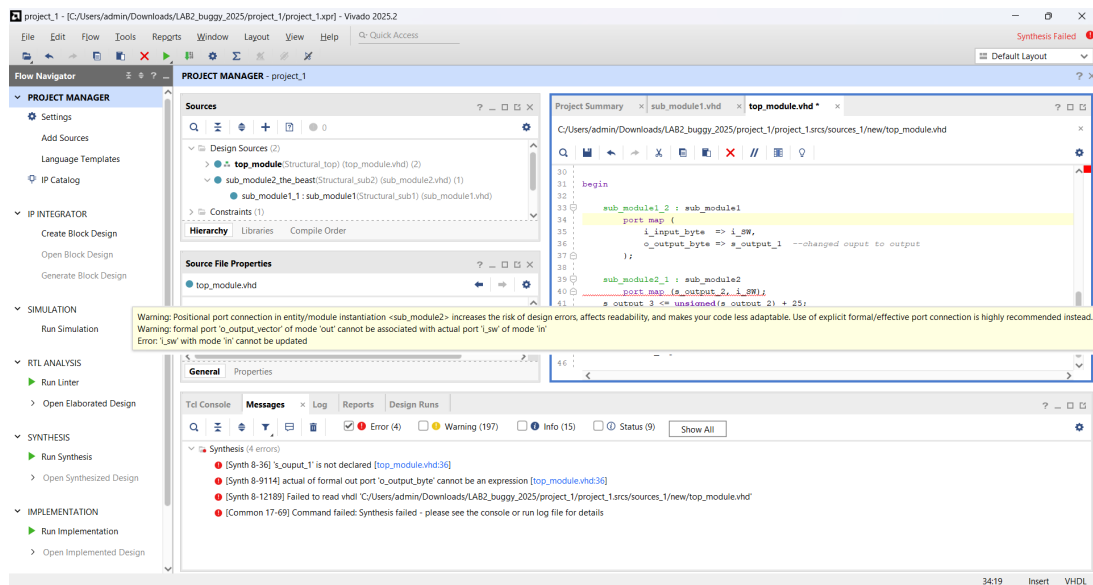


Figure 2: Typo fixed, other errors remain

Port Map Order Error: An error was found in the structural port mapping within the top_module.vhd. The signals were not being passed in the correct sequence, which was resolved by changing the mapping order.

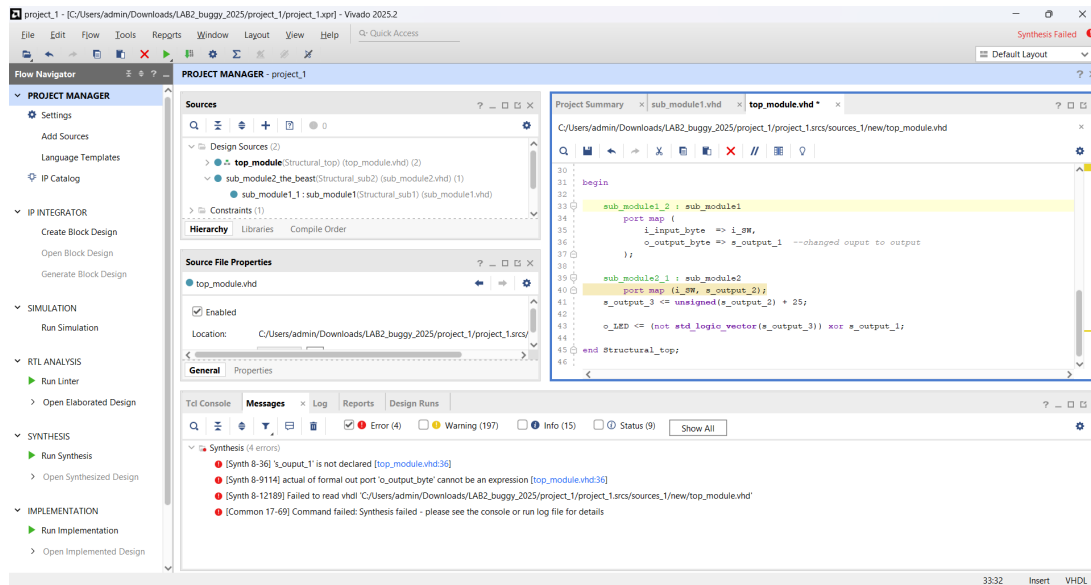


Figure 3: Order error fixed

After fixing the initial errors, synthesis was run again and completed successfully. Later, the attempts to run Implementation revealed additional errors:

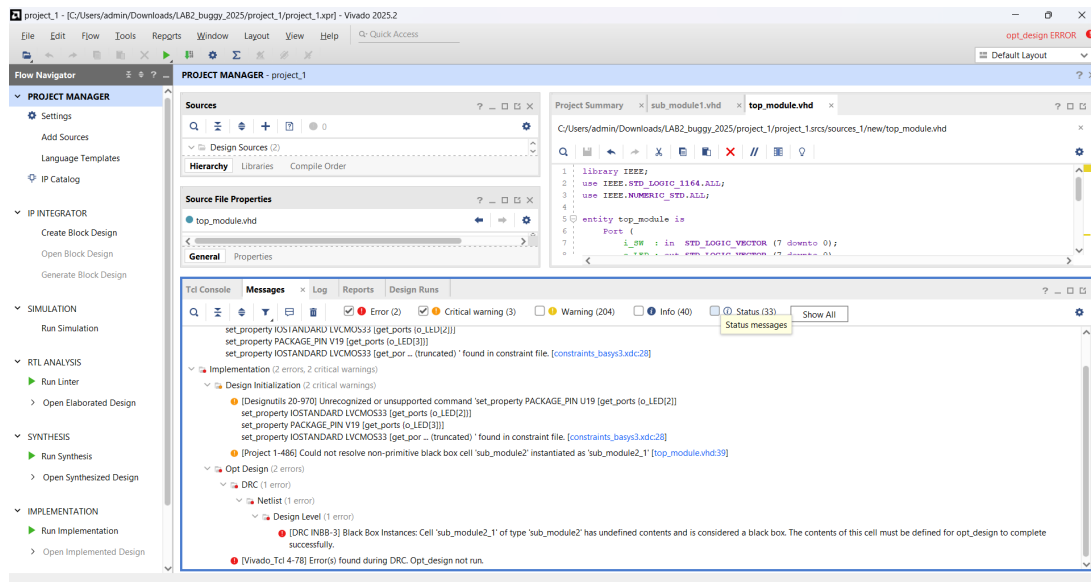
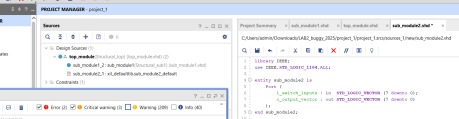


Figure 4: Implementation Errors



The screenshot displays the Visual Studio Code interface with the following components:

- Top Bar:** Shows the file explorer, search, and run and debug buttons. The title bar indicates the file path: `C:\Users\user\Documents\PROJECT_MANAGER\src\main.cpp`.
- Project Manager:** Displays the project structure for `PROJECT_MANAGER`, showing a `src` folder.
- Explorer:** Shows the `src` folder containing `main.cpp` and `main.h`.
- Run and Debug Console:** Shows the output of the program, including the text `Hello, World!` and `Hello, World!`.
- Code Editor:** Displays the `main.cpp` file with the following code:


```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello, World!" << endl;
    return 0;
}
```

Figure 5: Fixing naming inconsistency

The screenshot displays the Visual Studio IDE interface. On the left, the 'PROJECT MANAGER' tab is active, showing a list of projects including 'PROJECT MANAGER' and 'PROJECT MANAGER'. Below this, the 'SOLUTION EXPLORER' shows the 'PROJECT MANAGER' project with files like 'PROJECT MANAGER' and 'PROJECT MANAGER'. The main window shows the 'PROJECT MANAGER' project with the file 'PROJECT MANAGER' open, displaying C# code for a 'PROJECT MANAGER' class. The code includes a 'PROJECT MANAGER' class with a 'PROJECT MANAGER' method. The code is as follows:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace PROJECT MANAGER
{
    class PROJECT MANAGER
    {
        // ... (code omitted) ...
    }
}

```

The screenshot shows the Visual Studio IDE with the following components:

- Top Bar:** Displays the file path `intellisense\src\... \CSharpApp2\Program.cs` and the file name `Program.cs`.
- Left Sidebar:** Contains the **Project Explorer** showing the project structure:
 - Project Explorer:** Shows the project `CSharpApp2` with sub-items `bin`, `obj`, `Program.cs`, and `Properties`.
 - Solution Explorer:** Shows the solution `CSharpApp2` with sub-items `bin`, `obj`, `Program.cs`, and `Properties`.
- Code Editor:** Displays the source code for `Program.cs`. The code defines a `Person` class and a `Main` method. The `Person` class is highlighted in blue.


```
using System;

namespace CSharpApp2
{
    class Program
    {
        static void Main()
        {
            Person person = new Person("John", 30, "Male");
            Console.WriteLine(person.Name);
            Console.WriteLine(person.Age);
            Console.WriteLine(person.Gender);
        }
    }
}

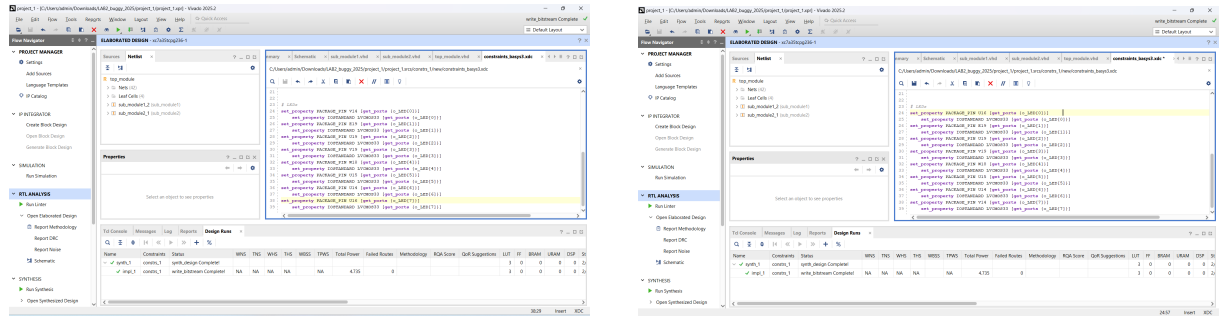
public class Person
{
    string Name;
    int Age;
    string Gender;

    public Person(string name, int age, string gender)
    {
        Name = name;
        Age = age;
        Gender = gender;
    }
}
```
- Output Window:** Shows the output of the program:


```
John
30
Male
```
- Properties Window:** Shows the properties of the selected item, which is the `Person` class. The `Language` property is set to `C#`.

Figure 6: Fixing constraints syntax

After fixing all the implementation errors, a final check for the constraints was done. This check revealed that the physical pins V14 (Led 7) and U16 (Led 0) were swapped. This was corrected to ensure the output signals appeared on the intended LEDs.



(a) Constraint Swapped

(b) Constraint Fixed

Figure 7: Fixing pin assignment

With all the errors resolved, the Generate Bitstream command was used to create the final configuration file for the FPGA hardware. However, for more correctly testing, first simulation (task 4) was done, then hardware testing (task 3) was done.

4 Task 4 Behavioral Simulation

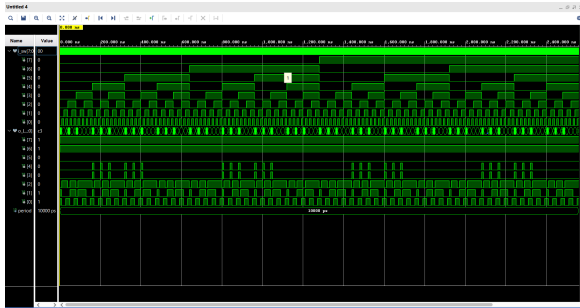
To verify the logic, a testbench was authored to cycle through all possible switch combinations. The simulation successfully generated a waveform that matched the expected truth tables for the AND, OR, and XOR gates.

5 Task 3, Hardware Demonstration

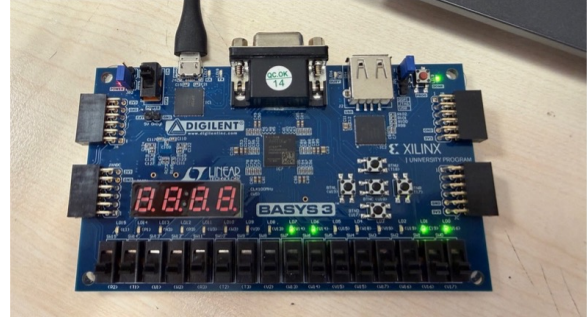
After successful debugging and simulation, the design was programmed onto the Basys 3 board. This step involved a physical comparison between the hardware's behavior and the simulation results to ensure a successful implementation. The programmed FPGA was tested by toggling the switches and observing the green LEDs. Each state was cross-referenced with the simulation checks performed in Task 4.

6 Simulation and Hardware Results

The board's behavior was found to be in complete agreement with the simulation waveforms. This confirmed that the constraint bugs and the logic bugs in the VHDL code were fully resolved.

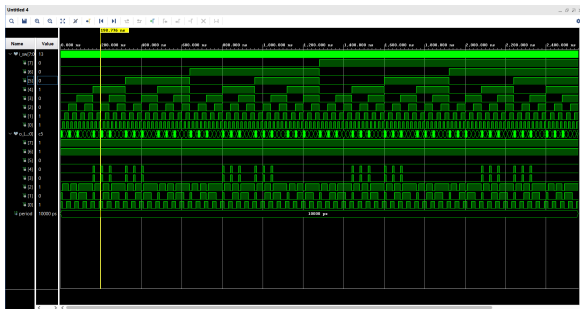


(a) Simulation results 1

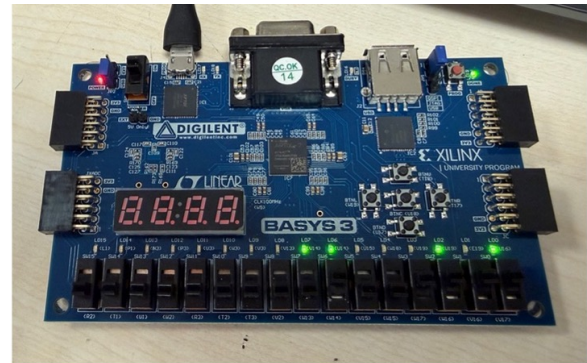


(b) Board confirmation 1

Figure 8: Comparison 1



(a) Simulation results 2



(b) Board confirmation 2

Figure 9: Comparison 2

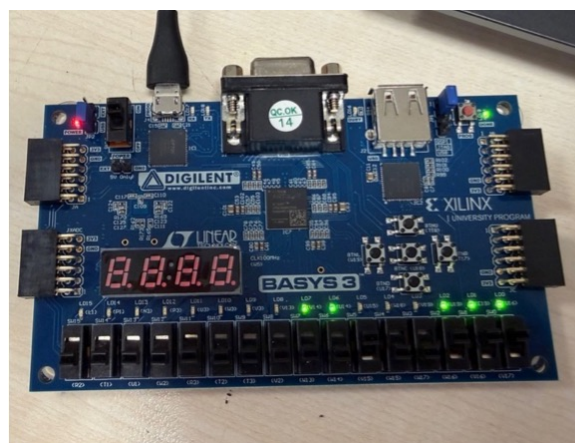
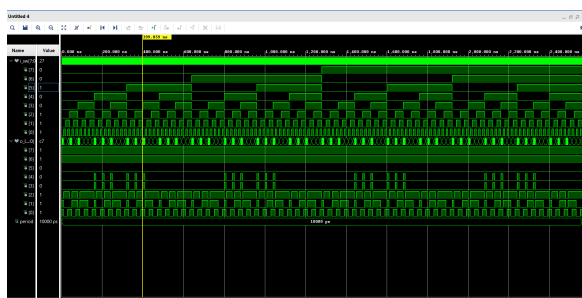


Figure 10: Comparison 3

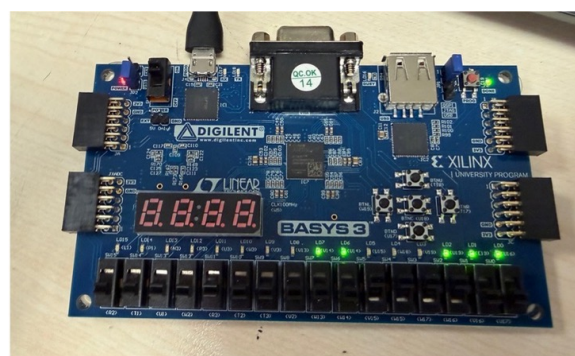
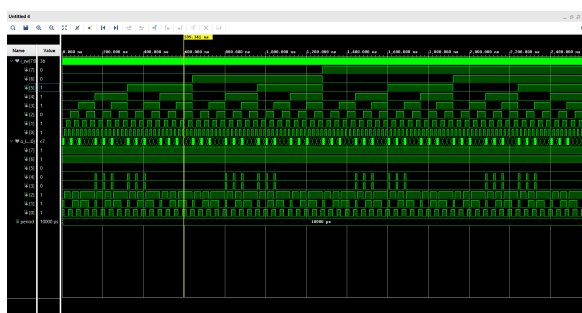
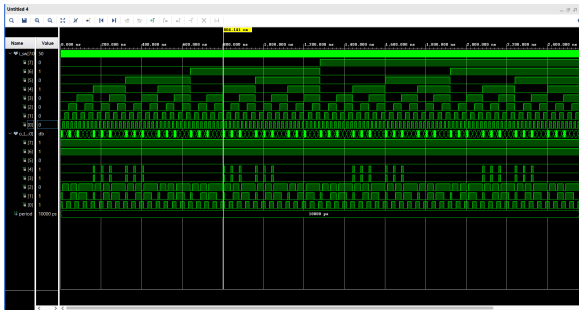
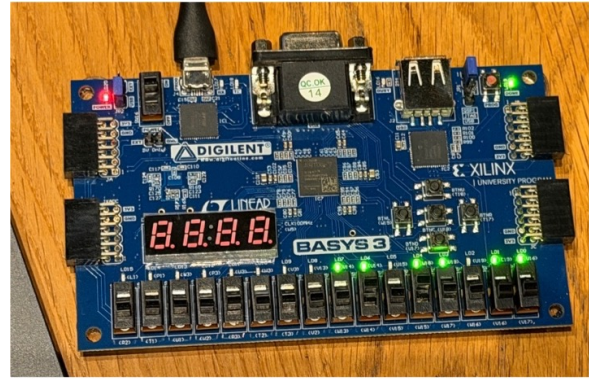


Figure 11: Comparison 4



(a) Simulation results 5



(b) Board confirmation 5

Figure 12: Comparison 5

7 Task 5, Schematic Analysis

Ran all three schematics.

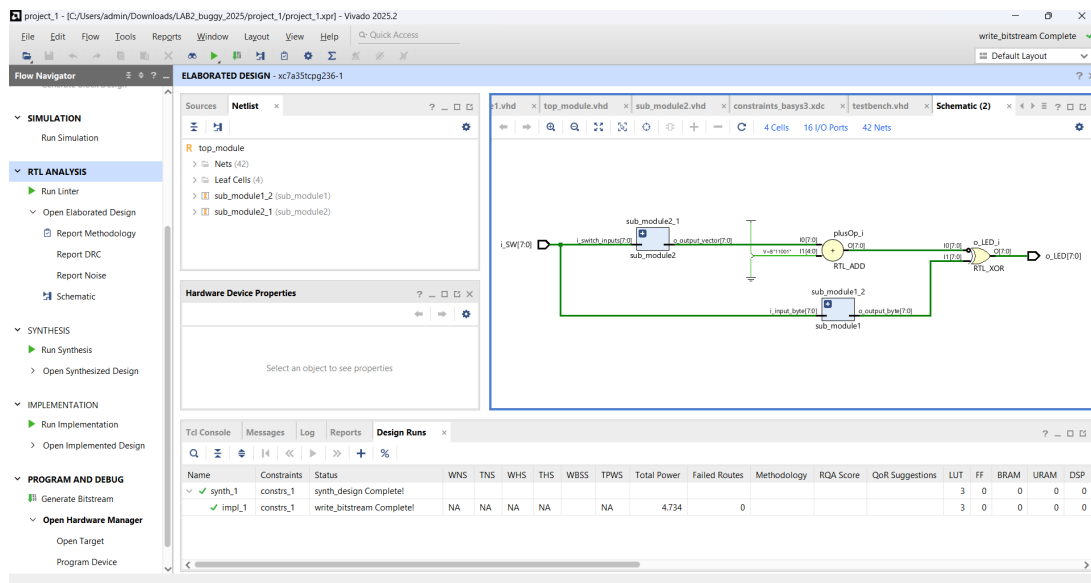


Figure 13: RTL Schematic

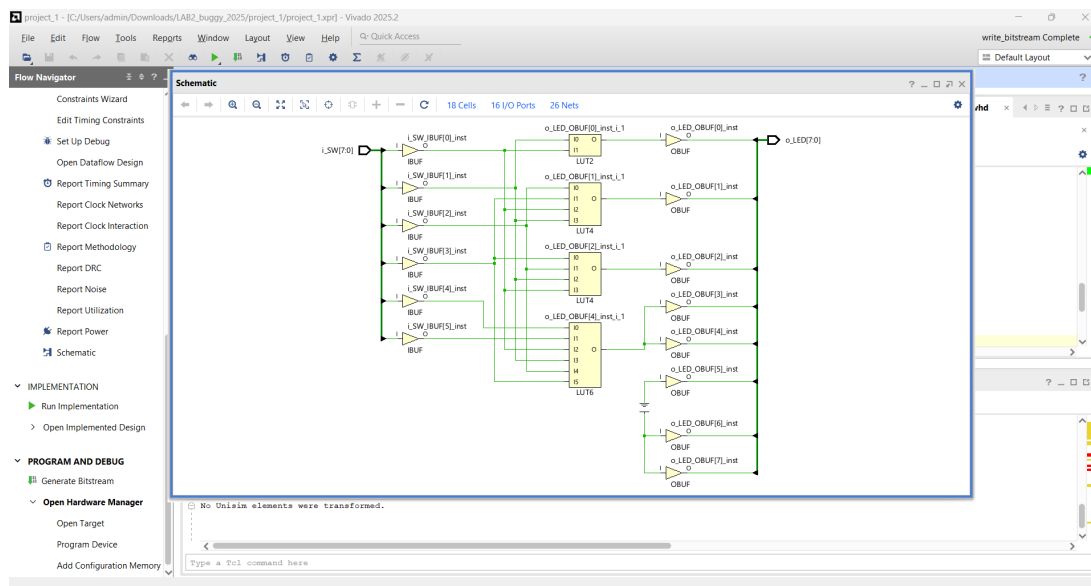


Figure 14: Synthesis Schematic

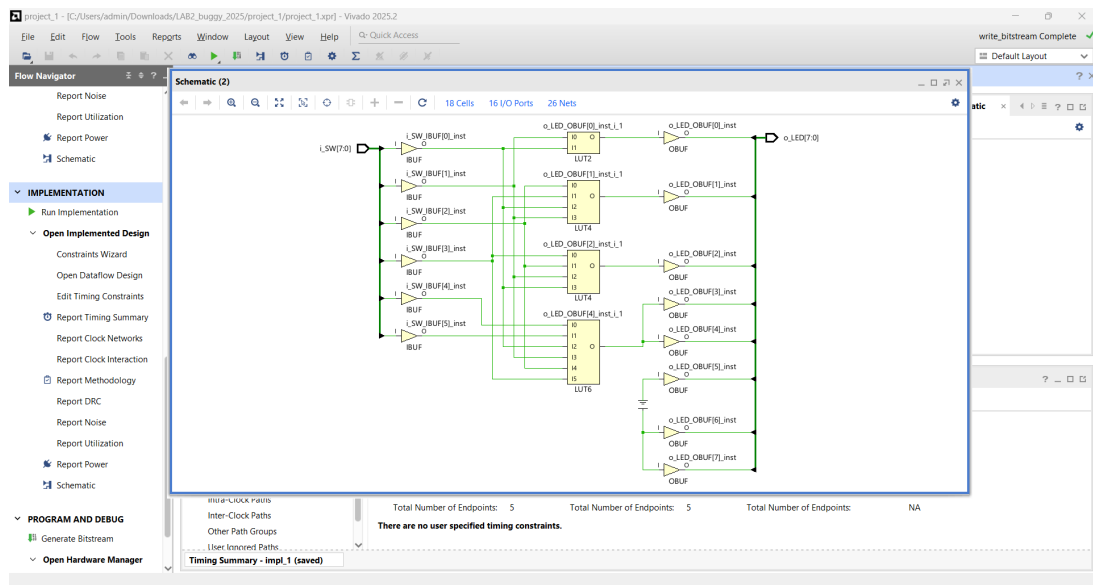


Figure 15: Implementation Schematic

8 Task 6, All Codes

8.1 top_module.vhd

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.NUMERIC_STD.ALL;
4 entity top_module is
5     Port (
6         i_SW  : in  STD_LOGIC_VECTOR (7 downto 0);
7         o_LED : out STD_LOGIC_VECTOR (7 downto 0)
8     );
9 end top_module;
10 architecture Structural_top of top_module is
11     component sub_module1 is
12         port (
13             i_input_byte  : in  STD_LOGIC_VECTOR (7 downto 0);
14             o_output_byte : out STD_LOGIC_VECTOR (7 downto 0)
15         );
16     end component sub_module1;
17     component sub_module2 is
18         port (
19             i_switch_inputs : in  STD_LOGIC_VECTOR (7 downto 0);
20             o_output_vector : out STD_LOGIC_VECTOR (7 downto 0)
21         );
22     end component sub_module2;
23     signal s_output_1, s_output_2 : STD_LOGIC_VECTOR(7 downto 0) := (others
24     => '0');
25     signal s_output_3              : unsigned(7 downto 0)          := (others
26     => '0');
27 begin
28     sub_module1_2 : sub_module1
29         port map (
30             i_input_byte  => i_SW,
31             o_output_byte => s_output_1  --changed ouput to output
32         );
33     sub_module2_1 : sub_module2
34         port map (i_SW, s_output_2);
35     s_output_3 <= unsigned(s_output_2) + 25;
36     o_LED <= (not std_logic_vector(s_output_3)) xor s_output_1;
37 end Structural_top;
```

8.2 sub_module1.vhd

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 entity sub_module1 is
4     Port (
5         i_input_byte  : in  STD_LOGIC_VECTOR (7 downto 0);
6         o_output_byte : out STD_LOGIC_VECTOR (7 downto 0)
7     );
8 end sub_module1;
9 architecture Structural_sub1 of sub_module1 is
10 begin
11     o_output_byte(0) <= i_input_byte(0) and i_input_byte(1);
12     o_output_byte(1) <= i_input_byte(2) or i_input_byte(3);
```

```

13 o_output_byte(2)          <= i_input_byte(4) xor i_input_byte(5);  --My
   id is 22502100
14 o_output_byte(5 downto 3) <= "010";
15 o_output_byte(7 downto 6) <= (others => '0');
16 end Structural_sub1;

```

8.3 sub_module2.vhd

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 entity sub_module2 is
4     Port (
5         i_switch_inputs : in  STD_LOGIC_VECTOR (7 downto 0);
6         o_output_vector : out STD_LOGIC_VECTOR (7 downto 0)
7     );
8 end sub_module2;
9 architecture Structural_sub2 of sub_module2 is
10     component sub_module1 is
11         port (
12             i_input_byte : in  STD_LOGIC_VECTOR (7 downto 0);
13             o_output_byte : out STD_LOGIC_VECTOR (7 downto 0)
14         );
15     end component sub_module1;
16     signal s_inv_input : STD_LOGIC_VECTOR(7 downto 0) := (others => '0');
17 begin
18     s_inv_input <= not i_switch_inputs;
19     sub_module1_1 : sub_module1
20         port map (
21             i_input_byte => s_inv_input,
22             o_output_byte => o_output_vector
23         );
24 end Structural_sub2;

```

8.4 Constraints_basys3.xdc

```

1 # Constraint file sets which input/output in the VHDL code connects to
   which pin in FPGA.
2 # Three characters after PACKAGE_PIN gives the FPGA pin which is also
   written next to all LEDs/Switches on the board.
3 # The second line (IOSTANDARD LVCMOS33) sets the voltage standard for the
   pins which is constant and always the same for BASYS3.
4 set_property PACKAGE_PIN V17 [get_ports {i_SW[0]}]
5 set_property IOSTANDARD LVCMOS33 [get_ports {i_SW[0]}]
6 set_property PACKAGE_PIN V16 [get_ports {i_SW[1]}]
7 set_property IOSTANDARD LVCMOS33 [get_ports {i_SW[1]}]
8 set_property PACKAGE_PIN W16 [get_ports {i_SW[2]}]
9 set_property IOSTANDARD LVCMOS33 [get_ports {i_SW[2]}]
10 set_property PACKAGE_PIN W17 [get_ports {i_SW[3]}]
11 set_property IOSTANDARD LVCMOS33 [get_ports {i_SW[3]}]
12 set_property PACKAGE_PIN W15 [get_ports {i_SW[4]}]
13 set_property IOSTANDARD LVCMOS33 [get_ports {i_SW[4]}]
14 set_property PACKAGE_PIN V15 [get_ports {i_SW[5]}]
15 set_property IOSTANDARD LVCMOS33 [get_ports {i_SW[5]}]
16 set_property PACKAGE_PIN W14 [get_ports {i_SW[6]}]
17 set_property IOSTANDARD LVCMOS33 [get_ports {i_SW[6]}]

```

```

18 set_property PACKAGE_PIN W13 [get_ports {i_SW[7]}]
19   set_property IOSTANDARD LVCMOS33 [get_ports {i_SW[7]}]
20
21 # LEDs
22 set_property PACKAGE_PIN U16 [get_ports {o_LED[0]}]
23   set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[0]}]
24 set_property PACKAGE_PIN E19 [get_ports {o_LED[1]}]
25   set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[1]}]
26 set_property PACKAGE_PIN U19 [get_ports {o_LED[2]}]
27   set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[2]}]
28 set_property PACKAGE_PIN V19 [get_ports {o_LED[3]}]
29   set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[3]}]
30 set_property PACKAGE_PIN W18 [get_ports {o_LED[4]}]
31   set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[4]}]
32 set_property PACKAGE_PIN U15 [get_ports {o_LED[5]}]
33   set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[5]}]
34 set_property PACKAGE_PIN U14 [get_ports {o_LED[6]}]
35   set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[6]}]
36 set_property PACKAGE_PIN V14 [get_ports {o_LED[7]}]
37   set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[7]}]

```

8.5 testbench.vhd

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.NUMERIC_STD.ALL;
4 entity testbench is
5 -- Testbench has no inputs or outputs
6 end testbench;
7 architecture Behavioral of testbench is
8 -- Component Declaration for the Unit Under Test (UUT)
9 component top_module
10 Port (
11 i_sw : in STD_LOGIC_VECTOR (7 downto 0);
12 o_LED : out STD_LOGIC_VECTOR (7 downto 0)
13 );
14 end component;
15 -- Inputs
16 signal i_sw : std_logic_vector(7 downto 0) := (others => '0');
17 -- Outputs
18 signal o_LED : std_logic_vector(7 downto 0);
19 -- Clock period definitions (just for timing)
20 constant period : time := 10 ns;
21 begin
22 -- Instantiate the Unit Under Test (UUT)
23 uut: top_module PORT MAP (
24 i_sw => i_sw,
25 o_LED => o_LED
26 );
27 -- Stimulus process
28 stim_proc: process
29 begin
30 -- Loop to test all 256 combinations (00000000 to 11111111)
31 for i in 0 to 255 loop
32 i_sw <= std_logic_vector(to_unsigned(i, 8));
33 wait for period;
34 end loop;

```

```
35 wait; -- Stop simulation
36 end process;
37 end Behavioral;
```

9 Conclusion

In this laboratory experiment, the fundamentals of digital design using VHDL and the Vivado IDE were successfully explored and applied. The primary objective of designing, debugging, and implementing a combinational logic circuit on the Basys 3 FPGA was met through a systematic workflow. By actively troubleshooting six distinct errors—ranging from VHDL syntax typos to physical constraint mapping—practical debugging skills were developed. The process highlighted the critical importance of accurate port mapping and constraint definitions in bridging software code with physical hardware. Furthermore, the creation of a testbench for behavioral simulation proved to be an essential step, allowing for the verification of the custom logic (AND, OR, XOR gates) prior to hardware deployment. Ultimately, the physical implementation on the Basys 3 board confirmed the theoretical design, with the LED outputs perfectly matching the expected truth tables for the assigned student ID logic. This lab provided valuable experience in the complete FPGA design flow, from code composition and simulation to bitstream generation and hardware verification, laying a strong foundation for future digital system design projects.